Mini Compiler Design and Implementation:
Phase 2 - Grammar Design and Syntactic Validation

Koushik-2023A7PS0129U , Siddhi-2023A7PS0020U,
Prisha-2023A7PS0273U , Aakanksh- 2023A7PS0143U

Question 2: Grammar Design and Syntactic Validation (7 Marks)
Date of Evaluation: 16/03/2026 - 20/03/2026

Construct a complete Context-Free Grammar (CFG) that generates the prescribed core language, including:

- Declarations
- Assignment statements
- Expressions
- Boolean expressions
- if–else statements
- while loops
- Block structures The grammar must be capable of generating the entire evaluation program without modification.

You must then demonstrate syntactic validation by:

- Showing leftmost derivation for at least one non-trivial statement from the evaluation program.
- Showing rightmost derivation for at least one non-trivial statement.
- Constructing the corresponding parse tree. The syntax analyzer must consume the token stream produced by your lexical analyzer.

---

1. Introduction

This phase involves the design and implementation of a **Syntax Analyzer (Parser)** for a simplified programming language. The syntax analyzer works together with the **lexical analyzer** developed in the previous phase. The lexical analyzer scans the input program and converts it into a stream of tokens, which are then processed by the syntax analyzer.

The main objective of this phase is to define a **Context-Free Grammar (CFG)** that describes the syntax of the core language. The grammar supports common programming constructs such as **variable declarations, assignment statements, arithmetic expressions, boolean expressions, conditional statements (if–else), while loops, and block structures**.

The syntax analyzer is implemented using **Bison**, while the lexical analyzer is implemented using **Flex**. The parser validates whether the token stream generated by the lexical analyzer conforms to the defined grammar rules. If the syntax is correct, the program confirms successful syntactic validation; otherwise, it reports an error.

To demonstrate syntactic validation, **leftmost derivation, rightmost derivation, and a parse tree** are constructed for a non-trivial statement from the evaluation program.

2. Formal Syntax Specification

The following table defines the language's Syntax structure using Context Free Grammar (CFG).

Non-Terminal	Production Rule
Program	Program → UnitList
UnitList	UnitList → UnitList Unit | ε
Unit	Unit → Declaration | Statement
Declaration	Declaration → Type ID ;
Type	Type → int | float
Statement	Statement → AssignmentStmt | IfStmt | WhileStmt | PrintStmt | CompoundStmt
CompoundStmt	CompoundStmt → { UnitList }
AssignmentStmt	AssignmentStmt → ID = Expression ;

IfStmt	IfStmt → if (Condition) Statement | if (Condition) Statement else Statement
WhileStmt	WhileStmt → while (Condition) Statement
PrintStmt	PrintStmt → print (Expression) ;
Expression	Expression → Expression + Expression
	Expression → Expression - Expression
	Expression → Expression * Expression
	Expression → Expression / Expression
	Expression → Expression % Expression
	Expression → (Expression)
	Expression → ID
	Expression → ICONST
	Expression → FCONST
Condition	Condition → Expression == Expression
	Condition → Expression != Expression
	Condition → Expression < Expression
	Condition → Expression > Expression

	Condition → Expression <= Expression
	Condition → Expression >= Expression
	Condition → (Condition)
	Condition → ! Condition
	Condition → Condition && Condition
	Condition → Condition || Condition

3. Implementation (C Source Code)

compi.l

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "y.tab.h"

int line = 1;
%}

DIGIT      [0-9]
LETTER     [A-Za-z_]
ID         {LETTER}({LETTER}|{DIGIT})*
ICONST     0|[1-9]{DIGIT}*
FCONST     {DIGIT}+"."{DIGIT}+

%%

"int"        { return INT; }
"float"      { return FLOAT; }
"if"         { return IF; }
"else"       { return ELSE; }
"while"      { return WHILE; }
```

```
"print"      { return PRINT; }

{FCONST}     { yylval.fval = atof(yytext); return FCONST; }
{ICONST}     { yylval.ival = atoi(yytext); return ICONST; }

{ID}         { yylval.sval = strdup(yytext); return ID; }

"=="         { return EQ; }
"!="         { return NE; }
"<="         { return LE; }
">="         { return GE; }
"&&"         { return AND; }
"||"         { return OR; }

"<"          { return '<'; }
">"          { return '>'; }
"="          { return '='; }

"+"          { return '+'; }
"-"          { return '-'; }
"*"          { return '*'; }
"/"          { return '/'; }
"%"          { return '%'; }
"!"          { return '!'; }

"("          { return '('; }
")"          { return ')'; }
"{"          { return '{'; }
"}"          { return '}'; }
";"          { return ';'; }
","          { return ','; }

[ \t]+       { }
\n           { line++; }

.            { printf("Lexical error at line %d: %s\n", line, yytext); }

%%

int yywrap() { return 1; }
```

Compi.y

```
%{
#include <stdio.h>
#include <stdlib.h>

extern int line;
int yylex(void);
void yyerror(char *s);
%}

%union {
    int ival;
    float fval;
    char* sval;
}

%token <sval> ID
%token <ival> ICONST
%token <fval> FCONST
%token INT FLOAT IF ELSE WHILE PRINT
%token EQ NE LE GE AND OR

/* Operator Precedence and Associativity */
%left OR
%left AND
%left EQ NE LE GE '<' '>'
%left '+' '-'
%left '*' '/' '%'
%right '!'
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE

%%

/* The program is a series of units (declarations or statements) */
program:
    unit_list
    ;

unit_list:
    unit_list unit
    | /* empty */
    ;

unit:
    declaration
```

```
    | statement
    ;

declaration:
    type ID ';' { printf("Line %d: Syntactic Validation [Declaration: %s]\n",
line, $2); free($2); }
    ;

type:
    INT | FLOAT
    ;

statement:
    assignment_stmt
    | if_stmt
    | while_stmt
    | print_stmt
    | compound_stmt
    ;

/* Allows declarations and statements to mix inside braces */
compound_stmt:
    '{' unit_list '}'
    ;

assignment_stmt:
    ID '=' expression ';' { printf("Line %d: Syntactic Validation [Assignment
to %s]\n", line, $1); free($1); }
    ;

if_stmt:
    IF '(' condition ')' statement %prec LOWER_THAN_ELSE
    | IF '(' condition ')' statement ELSE statement { printf("Line %d:
Syntactic Validation [If-Else Block]\n", line); }
    ;

while_stmt:
    WHILE '(' condition ')' statement { printf("Line %d: Syntactic Validation
[While Loop]\n", line); }
    ;

print_stmt:
    PRINT '(' expression ')' ';' { printf("Line %d: Syntactic Validation [Print
Statement]\n", line); }
    ;
```

```
expression:
    expression '+' expression
    | expression '-' expression
    | expression '*' expression
    | expression '/' expression
    | expression '%' expression
    | '(' expression ')'
    | ID { free($1); }
    | ICONST
    | FCONST
    ;

condition:
    expression EQ expression
    | expression NE expression
    | expression '<' expression
    | expression '>' expression
    | expression LE expression
    | expression GE expression
    | '(' condition ')'
    | '!' condition
    | condition AND condition
    | condition OR condition
    ;

%%

int main() {
    if (yyparse() == 0) {
        printf("\nRESULT: Syntactic Validation Successful.\n");
    } else {
        printf("\nRESULT: Syntactic Validation Failed.\n");
    }
    return 0;
}

void yyerror(char *s) {
    fprintf(stderr, "Syntax Error at line %d: %s\n", line, s);
}
```

4. Evaluation and Error Reporting

4.1 Input Source Code (Snippet)

```
int a;
int b;
int sum;
float avg;
a = 2 * (3 + 4);
b = 15;
sum = 0;
while (a < b && b != 0) {
 int temp;
 temp = a * 2;
 if ((temp % 3 == 0) || (a > 5)) {
 sum = sum + temp;
 } else {
 sum = sum - 1;
 }
 a = a + 1;
}
avg = sum / (b - a);
if (!(avg < 5.0)) {
 print(sum);
} else {
 print(avg);
}
```

4.2 Syntax Error Analysis

4.3 Final Syntax Identifications

Line No	Code	Syntax Found
1	int a;	Declaration a;
2	int b;	Declaration b;
3	int sum;	Declaration sum;
4	float avg;	Declaration avg;
5	a = 2 * (3 + 4);	Assignment a;

6	b = 15;	Assignment b;
7	sum = 0;	Assignment sum;
8	while (a < b && b != 0)	While Loop condition;
9	int temp;	Declaration temp;
10	temp = a * 2;	Assignment temp;
11	if ((temp % 3 == 0) || (a > 5))	If condition;
12	sum = sum + temp;	Assignment sum;
13	} else {	Else branch;
14	sum = sum - 1;	Assignment sum;
15	}	If-Else Block validated;
16	a = a + 1;	Assignment a;
17	}	While Loop;
18	avg = sum / (b - a);	Assignment avg
19	if (!(avg < 5.0))	If condition
20	print(sum);	Print Statement
21	} else {	Else branch
22	print(avg);	Print Statement
23	}	If-Else Block validated

5. Syntactic Validation

Statement: a = 2 * (3 + 4);

5.1 Leftmost Derivation

<statement>
⇒ <assignment_stmt>
⇒ ID = <expression> ;
⇒ a = <expression> ;
⇒ a = <expression> * <expression> ;
⇒ a = ICONST * <expression> ;
⇒ a = 2 * <expression> ;
⇒ a = 2 * (<expression>) ;
⇒ a = 2 * (<expression> + <expression>) ;
⇒ a = 2 * (ICONST + <expression>) ;
⇒ a = 2 * (3 + <expression>) ;
⇒ a = 2 * (3 + ICONST) ;
⇒ a = 2 * (3 + 4) ;

5.2 Rightmost Derivation

<statement>
⇒ <assignment_stmt>
⇒ ID = <expression> ;
⇒ ID = <expression> * <expression> ;
⇒ ID = <expression> * (<expression>) ;
⇒ ID = <expression> * (<expression> + <expression>) ;
⇒ ID = <expression> * (<expression> + 4) ;
⇒ ID = <expression> * (3 + 4) ;
⇒ ID = 2 * (3 + 4) ;
⇒ a = 2 * (3 + 4) ;

5.3 Parse Tree